

International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 7 – Inheritance

Date: 06.10.2026

Semester: B.Tech 3rd Semester

Group: B2

Instructions

- Write and execute the C++ program for each problem.
- Use appropriate classes, objects, constructors, and inheritance concepts wherever required.
- Follow the inheritance structure specified in each question.
- Display clear and meaningful output.
- Save all programs in your assigned GitHub repository.
- The repository submission must be completed by 12:00 AM on the same day to receive marks.
- Submit the lab record in the prescribed format.

1. Employee Salary – Multilevel Inheritance

Create a base class `Employee` containing `name` and `basicSalary`. Derive a class `Developer` containing `experience`, and then derive a class `SeniorDeveloper` containing `projectBonus`.

Calculate the final salary using:

$$\text{Final Salary} = \text{Basic Salary} + \text{Experience Bonus} + \text{Project Bonus}$$

where

$$\text{Experience Bonus} = 5\% \times \text{Basic Salary} \times \text{Experience}.$$

Use constructors to initialize the data members and display the final salary.

Hint: The inheritance should follow the structure `Employee` → `Developer` → `SeniorDeveloper`.

2. Student Result – Function Overriding

Create a base class `Student` containing `name`, `rollNo`, and a function `calculateResult()`. Derive two classes, `RegularStudent` and `ScholarshipStudent`.

For `RegularStudent`, calculate the total marks normally. For `ScholarshipStudent`, add 5 bonus marks to the total.

Override `calculateResult()` in both derived classes and display the final result.

Hint: The same function should behave differently in the two derived classes.

3. Vehicle Rental – Multilevel Inheritance

Create the following inheritance hierarchy:

Vehicle → Car → LuxuryCar

The class `Vehicle` should store the registration number and number of rental days. The class `Car` should store the daily rental rate. The class `LuxuryCar` should store an additional luxury charge per day.

Calculate and display the total rental cost.

$$\text{Total Cost} = (\text{Daily Rate} + \text{Luxury Charge}) \times \text{Rental Days}$$

4. Banking System – Hierarchical Inheritance

Create a base class `BankAccount` containing account number and balance.

Derive two classes, `SavingsAccount` and `CurrentAccount`.

For `SavingsAccount`, add interest to the balance.

For `CurrentAccount`, deduct a maintenance charge if the balance is below the specified minimum balance.

Display the updated balance for both account types.

Hint: Both derived classes should inherit from the same base class.

5. Student Performance – Multiple Inheritance

Create two base classes, `Academic` and `Sports`.

The class `Academic` should store marks in three academic subjects, while `Sports` should store sports marks.

Derive a class `StudentResult` from both classes.

Calculate:

$$\text{Total} = \text{Academic Marks} + \text{Sports Marks}$$

and

$$\text{Average} = \frac{\text{Total}}{4}.$$

Use constructors in all three classes and display the complete result.

Hint: The derived class should inherit data from both `Academic` and `Sports`.

6. Resolving Ambiguity in Multiple Inheritance

Create two base classes, `InternalExam` and `ExternalExam`. Both classes must contain a function named `display()`.

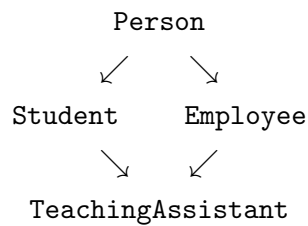
Derive a class `FinalResult` from both classes.

Call the appropriate `display()` function from `FinalResult` and resolve the ambiguity using the **scope resolution operator**.

Hint: Use the form `BaseClassName::functionName()` to specify which function should be called.

7. University Personnel – Hybrid Inheritance

Create the following inheritance hierarchy:



The class `Person` should contain name and age. The class `Student` should contain rollNo and CGPA, while `Employee` should contain employeeID and salary.

The class `TeachingAssistant` should display all the information.

Use **virtual inheritance** for `Student` and `Employee` so that only one copy of the `Person` members is inherited.

8. Hospital System – Protected Members

Create a base class `Patient` with the following protected data members:

- Patient name
- Patient ID
- Age

Derive a class `InPatient` containing room charges and number of days.

Calculate and display the total hospital bill.

Condition: The derived class must access the patient information through the inherited protected members. Do not make the members public.

9. Constructor Execution in Inheritance

Create the following inheritance hierarchy:

Person → Employee → Manager

Each class should contain its own data members and a constructor.

Create an object of `Manager`.

Display messages from each constructor so that the order of constructor execution is clearly visible:

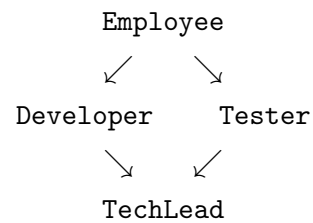
Person constructor
Employee constructor
Manager constructor

Finally, display all the initialized information.

Hint: Observe which constructor executes first when an object of the most-derived class is created.

10. Diamond Problem – Virtual Inheritance

Create the following inheritance hierarchy:



The class `Employee` should contain employee ID and name. The class `Developer` should contain the programming language, while `Tester` should contain the testing tool.

Use **virtual inheritance** so that `TechLead` contains only one copy of the `Employee` data.

Display all relevant information of the `TechLead`.

***Hint:** Use virtual base classes to solve the diamond inheritance problem.*